Repository Link:

https://github.com/abdomrieden/Algorithms-project

Introduction

This report presents solutions to the Triangular Triplet Problem using both iterative (non-recursive) and recursive approaches. A triangular triplet is defined as three elements in an array that satisfy the triangle inequality condition.

For three numbers (a), (b), and (c) to form a valid triangle:

a+b>c

After sorting the array in ascending order, checking this single condition becomes sufficient.

The objective of this project is to:

- Implement iterative and recursive algorithms
- Compare both approaches
- Analyze their time complexity
- Explain how sorting improves efficiency
- Implement a custom sorting algorithm without using built-in sorting functions

Problem Explanation

A triangle is valid only if:

- (a+b>c)
- (a+c>b)
- (b+c>a)

After sorting the array:

[

a \le b \le c

]

Only this condition is needed:

a+b>c

This reduces unnecessary comparisons and improves performance.

Merge Sort Explanation

Built-in sorting functions such as A.sort() were not used in this project. Instead, a custom Merge Sort algorithm was implemented.

Merge Sort works using the Divide and Conquer technique.

The algorithm:

1. Divides the array into two halves
2. Recursively sorts each half
3. Merges the sorted halves together

Example:

Original array:

8 3 5 1

Step 1 — Divide:

8 3 5 1

Step 2 — Divide again:

8 3 5 1

Step 3 — Merge sorted parts:

3 8 1 5

Final merge:

1 3 5 8

Merge Sort guarantees time complexity:

O(nlog n)

which makes it efficient for large datasets.

Full Program Code

```
def merge(left, right):

    result = []

    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] <= right[j]:
            result.append(left[i])
            i += 1

        else:
            result.append(right[j])
            j += 1

    while i < len(left):
        result.append(left[i])
        i += 1

    while j < len(right):
        result.append(right[j])
        j += 1

    return result
```

```
def merge_sort(arr):

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)


def iterative(A):

    if len(A) < 3:
        return 0

    A = merge_sort(A)

    for i in range(len(A) - 2):

        if A[i] + A[i + 1] > A[i + 2]:
            return 1

    return 0
```

```
def R_helper(A, i):

    if i >= len(A) - 2:
        return 0

    if A[i] + A[i + 1] > A[i + 2]:
        return 1

    return R_helper(A, i + 1)


def recursive(A):

    if len(A) < 3:
        return 0

    A = merge_sort(A)

    return R_helper(A, 0)


if __name__ == "__main__":

    print("TRIANGULAR TRIPLET DETECTOR")
    print("Enter positive numbers one by one.")
```

```
print("Type 'done' when finished.\\n")

numbers = []

while True:

    user_input = input("Enter number (or type 'done'): ")

    if user_input.lower() == "done":
        break

    try:

        num = float(user_input)

        if num <= 0:
            print("Only positive numbers greater than 0 are allowed.")
            continue

        numbers.append(num)

    except ValueError:
        print("Invalid input. Please enter a valid number.")

if len(numbers) < 3:
    print("\\nYou must enter at least 3 numbers.")
    exit()
```

```
arr_iter = numbers.copy()
arr_rec = numbers.copy()

result_iter = iterative(arr_iter)
result_rec = recursive(arr_rec)

sorted_array = merge_sort(numbers)

print("\\nOriginal Array:", numbers)
print("Sorted Array:", sorted_array)

if result_iter == 1:
    print("\\nTriangular triplet exists.")

else:
    print("\\nNo triangular triplet exists.")

print("Iterative Result:", result_iter)
print("Recursive Result:", result_rec)

input("\\nPress Enter to exit...")
```

Iterative Algorithm

Pseudocode

Algorithm Iterative(A)

```
if length(A) < 3:
    return 0

A = MergeSort(A)

for i from 0 to length(A)-3:

    if A[i] + A[i+1] > A[i+2]:
        return 1

return 0
```

Iterative Algorithm Analysis

The iterative solution performs two main operations:

1. Sorting Using Merge Sort

Merge Sort recursively divides the array into halves until each subarray contains one element.

The recurrence relation for Merge Sort is:

$T(n)=2T(n/2)+O(n)$

Using the Master Method:

- $(a = 2)$
- $(b = 2)$
- $(f(n) = O(n))$

Since:

[

n^{\log_b a}=n^{\log_2 2}=n

]

the complexity becomes:

T(n)=O(nlog n)

2. Iteration

The loop:

for i in range(len(A) - 2):

runs at most (n) times.

Each iteration performs a constant-time comparison:

A[i] + A[i+1] > A[i+2]

Therefore:

[

T(n)=O(n)

]

Total Iterative Complexity

Total complexity:

[

O(nlog n)+O(n)

]

Dominant term:

T(n)=O(nlog n)

Recursive Algorithm

Pseudocode

Algorithm Recursive(A, i)

if i >= length(A)-2:

```
    return 0

if A[i] + A[i+1] > A[i+2]:
    return 1

return Recursive(A, i+1)
```

Recursive Algorithm Analysis

The recursive solution also begins by sorting the array using Merge Sort:

O(nlog n)

After sorting, recursion checks one triplet at a time.

Recursive Relation

The recursive helper function follows:

T(n)=T(n-1)+O(1)

Where:

- (T(n-1)) represents the next recursive call
- (O(1)) represents the triangle comparison

Solving Using Iteration Method

Expand repeatedly:

[

T(n)=T(n-1)+c

]

[

$$T(n-1)=T(n-2)+c$$

Substitute:

$$T(n)=T(n-2)+2c$$

Continue expansion:

$$T(n)=T(n-k)+kc$$

When (k=n):

$$T(n)=T(0)+nc$$

Therefore:

$$T(n)=O(n)$$

Total Recursive Complexity

Sorting + recursion:

$$O(n\log n)+O(n)$$

Final complexity:

T(n)=O(n\log n)

Comparison Between Algorithms

Feature	Iterative	Recursive
Structure	Loop-based	Recursive calls
Time Complexity	O(n log n)	O(n log n)
Speed	Faster	Slightly slower

Discussion

Sorting is the main optimization in this project.

Without sorting, checking all possible triplets would require:

$O(n^3)$

because every possible combination of three elements would need testing.

After sorting, only consecutive triplets need checking, reducing complexity significantly to:

$O(n\log n)$

The iterative solution is simpler and faster in practice, while the recursive solution demonstrates recursion concepts effectively.

Conclusion

This project successfully implemented and analyzed both iterative and recursive solutions for the Triangular Triplet Problem using a custom Merge Sort implementation.

Final conclusions:

- Both algorithms correctly determine whether a valid triangle exists
- Both algorithms have overall complexity:

$O(n\log n)$

- Merge Sort is the dominant operation in both approaches
- The iterative solution is more practical and efficient
- The recursive solution is useful for demonstrating recursion principles

Therefore, the iterative approach is generally preferred for real-world applications and larger datasets.